A guide to the Qiskit circuit library

Qiskit · Follow
Published in Qiskit
10 min read · Oct 4, 2023

Listen Share

By Robert Davis, Julien Gacon, Abby Mitchell, and Luciano Bello

When it comes to quantum circuits, sometimes the "do it yourself" approach is best. Even for common subroutines like the quantum Fourier transform (QFT), a user may find that it's better to build their circuit by hand, so they can implement it in a way that works for their specific use case. But in many cases, the circuits we need are well-studied, and we can avoid having to reinvent the wheel (potentially creating bugs in the process) by simply using circuits that already exist. That's just one reason why we created the Qiskit circuit library.

The Qiskit circuit library is a collection of valuable, well-studied circuits and gates that Qiskit users can easily plug into their quantum circuits. These include useful tools for a wide array of quantum computing applications—including quantum machine learning, quantum chemistry, circuits for benchmarking, and even a number of circuits that are hard to simulate classically. Whether you're a complete beginner or a bona fide Qiskit wizard, we're sure you'll find something useful in our extensive circuit library module.

Why every Qiskit user should give the circuit library a closer look

Before we launched the circuit library a few years ago, users could only run quantum circuits they had constructed manually. If a user wanted to implement a phase estimation, a Fourier transform, or a variational circuit—really anything at all—they either had to build it themselves, or build a function that generated the circuit for them. In either case, the process was cumbersome, and documentation was limited and disorganized.

As the Qiskit community began to grow, we developed a clearer sense of which circuits were most important to users. Eventually, we began to organize these into the circuit library we have today. This library is designed to be a repository of building blocks—a collection of any and all circuits we think are valuable to users. Here, we define "valuable" circuits as those that are either (1) widely used for common quantum computing tasks, (2) difficult to simulate classically, or (3) useful for quantum hardware benchmarking.

For example, the quantum Fourier transform (QFT) is an essential building block in a number of quantum algorithms including quantum phase estimation and the famous Shor's algorithm. Now, if they really want to, a user can certainly run a QFT on quantum hardware by building the circuit manually. That might look something like this:

```
import numpy as np
from qiskit import QuantumCircuit

def build_qft(n):
   circuit = QuantumCircuit(n)
   for j in reversed(range(n)):
       circuit.h(j)
       for k in reversed(range(j)):
           circuit.cp(np.pi * 2. ** (k - j), j, k)

   for j in range(n // 2):
       circuit.swap(j, n - j - 1)

   return circuit

qft = build_qft(5)
```

But users who want to save time on this common quantum computing task can simply import the QFT from the circuit library and use it in their subroutines like so:

```
from qiskit.circuit.library import QFT

qft = QFT(5)
```

Most Qiskit users are aware the circuit library exists, but not everyone realizes just how useful it can be. That's why we suggest all users spend time familiarizing themselves with it. The circuit library provides essential tools for building quantum algorithms, and gives users the ability to write programs at a higher level of abstraction. Gates and other elements found in the circuit library are developed so that users can easily plug them into their own quantum circuits using the `QuantumCircuit.append()` method, meaning users shouldn't need to concern themselves with the lower level details of how those elements actually work.

The circuit library also provides tools that help users investigate properties of the quantum system itself, such as measuring quantum volume, and it offers useful information about the gates and circuits we use most often. It really is a library in the truest sense—in many cases; documentation for gates and other objects include links to a paper detailing the theory upon which they're based.

Now, let's take a look at some of the different kinds of circuits and gates that users will find in the Qiskit Circuit Library.

Variational quantum circuits ("N-local circuits")

The **N-local circuits sub-module** of the circuit library contains a selection of subclasses that serve as parameterized models in **variational algorithms**, such as the variational quantum eigensolver (VQE) algorithm and the quantum approximate optimization algorithm (QAOA).

In fields like chemistry and physics, these algorithms serve to approximate wave functions efficiently, which is a task that is generally hard for classical computers. Variational algorithms are also useful for problems in classical optimization and machine learning, where a quantum computer might enable a better optimization of the objective.

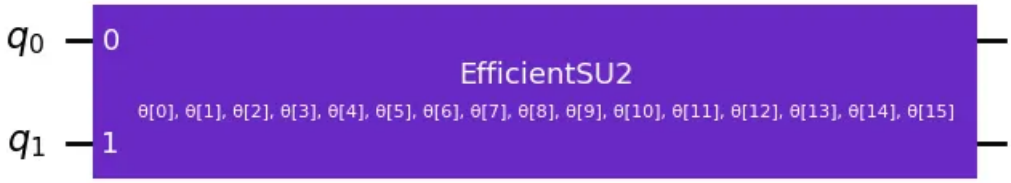
Let's take a look at an example of how to define a problem instance for a VQE algorithm, using the `EfficientSU2` ansatz circuit taken from the circuit library. In this example, we'll derive our Hamiltonian from a quantum chemistry problem:

```
from qiskit.quantum_info import SparsePauliOp

hamiltonian = SparsePauliOp.from_list(
    [("YZ", 0.3980), ("ZI", -0.3980), ("ZZ", -0.0113), ("XX", 0.1810)]
)
```

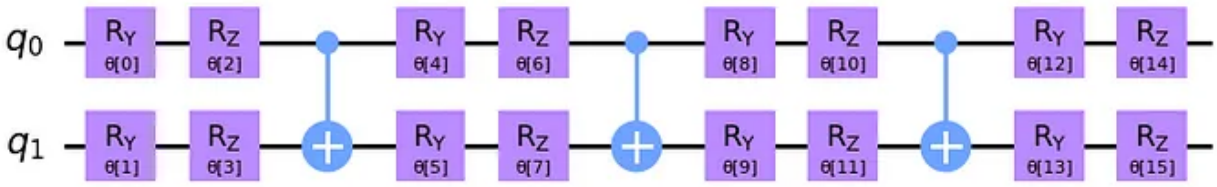
As mentioned above, our choice of ansatz is the EfficientSU2 circuit. By default, the `EfficientSU2` circuit linearly entangles qubits. This makes it ideal for use in quantum hardware with limited connectivity:

```
from qiskit.circuit.library import EfficientSU2

ansatz = EfficientSU2(hamiltonian.num_qubits)
ansatz.draw("mpl")
```

Here's what the circuit looks like:



Try printing with the following code to see how the circuit is structured!



Based on the previous figure, we know that our ansatz circuit is defined by a vector of parameters, θi, with the total number given by:

```
num_params = ansatz.num_parameters
print(num_params)
```

And in this case we get the output:

```
16
```

For an example on how to use the circuit library to run a VQE, see:
https://qiskit.org/ecosystem/algorithms/tutorials/01_algorithms_introduction.html

Data encoding circuits

Data encoding circuits enable the preparation of input data for QML applications. There are several techniques to encode classical data, or features, into a quantum state, for example:

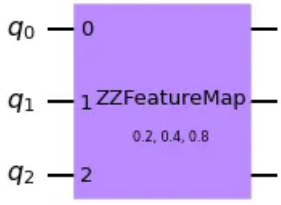
- Amplitude encoding: Encode the features into the amplitude of a basis state. This allows to store `2^n` numbers in a single state, but can be costly to implement
- Basis encoding: Encode an integer `k` by preparing the corresponding basis state `|k>`
- Angle encoding: Encode features as rotation angles in a variational quantum circuit

Deciding which of these approaches is best really depends upon the specifics of your application. On current quantum computers, however, we often use angle encoding.

One example is the ZZFeatureMap found in the circuit library:

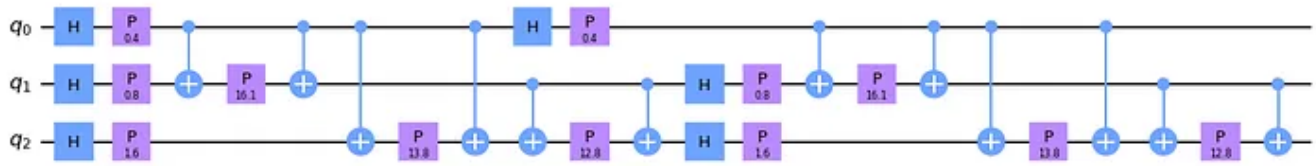

```
encoded = feature_map.assign_parameters(feature)
encoded.draw('mpl')
```

which prints



Try printing with the following code to see how the circuit is structured!

```
encoded.decompose().draw('mpl')
```



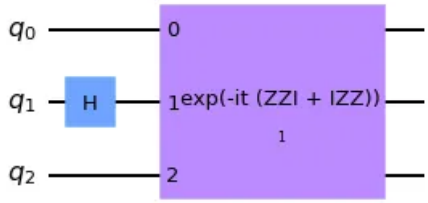
For an example of how to use data encoding circuits for classication, see: https://qiskit.org/ecosystem/machine-learning/tutorials/02_neural_network_classifier_and_regressor.html.

Time evolution circuits

Time evolution circuits allow us to evolve a quantum state in time. Physics researchers make use of them for a wide-array of applications, including investigations of thermalization properties or potential phase transitions of a system. Time evolution circuits are also a fundamental building block of chemistry wave functions—e.g., for unitary coupled-cluster trial states—and of the QAOA algorithm we use for optimization problems.

```
from qiskit.circuit.library import PauliEvolutionGate
from qiskit.circuit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp

hamiltonian = SparsePauliOp(["ZZI", "IZZ"])

# prepare an initial state with a Hamadard on the middle qubit
state = QuantumCircuit(3)
state.h(1)

evolution = PauliEvolutionGate(hamiltonian, time=1)

# evolve state by appending the evolution gate
state.append(evolution)

state.draw('mpl')
```

And in this case we get the output:



Benchmarking and complexity theory circuits

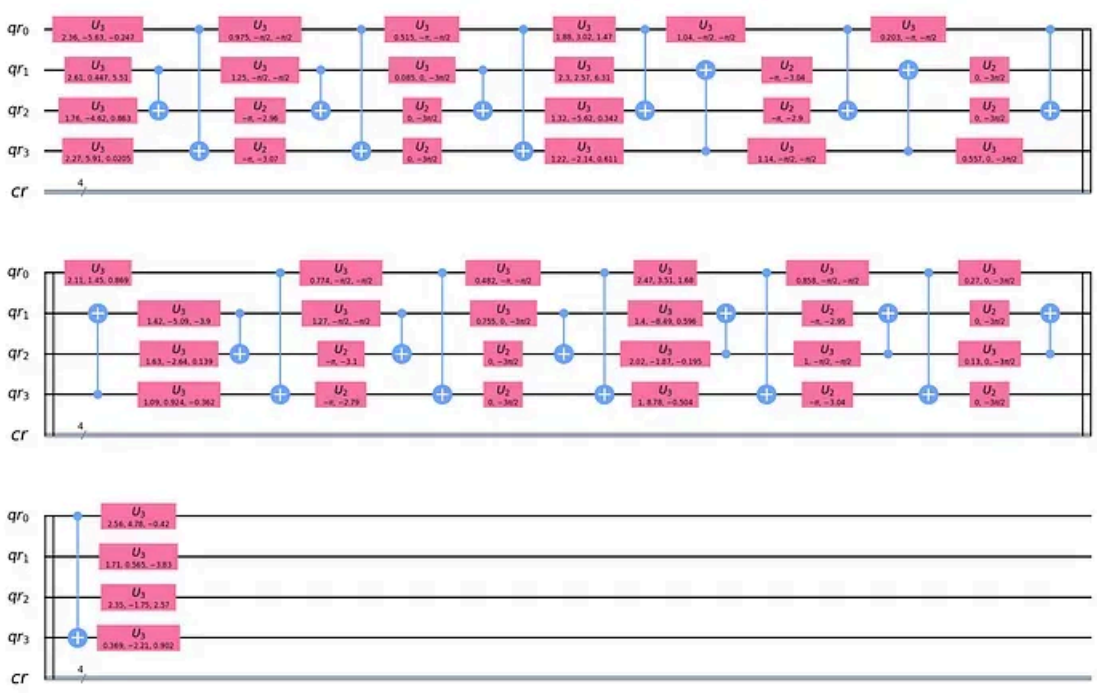
Benchmarking and complexity theory circuits are a vital tool for anyone working with quantum computers. They give us a sense of how well our hardware is actually working, and the true level of computational complexity involved in the problems we want to solve.

One of the most important examples of this is the `QuantumVolume` circuit, which allows us to quantify the largest random circuit of equal width and depth that a near-term quantum computer of relatively modest

size can successfully implement. This single metric can help us understand how well the quantum computer is doing in terms of fidelity of operations, qubit connectivity, gate set calibration, and circuit rewiring tool chains.

A quantum volume model circuit comprises layers of Haar random elements of SU(4) applied between corresponding pairs of qubits in a random bipartition. Here's an example of a quantum volume circuit built in Qiskit that runs on four qubits:

```
from qiskit.circuit.library import QuantumVolume

qv = QuantumVolume(4)
qv.decompose().decompose().draw('mpl')
```



Some quantum circuits have been shown to deliver advantage over classical algorithms. Many readers will already be familiar with some of the more famous examples of these circuits, which include Shor's algorithm for prime factorizationand the Grover search algorithm.

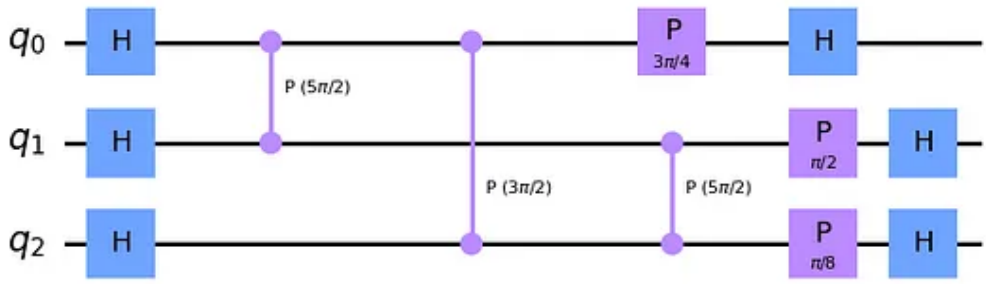
The circuit library contains circuits that are conjectured to be hard to simulate classically, such as the IQP circuit. (Learn more about the IQP circuit here.)

It also contains building blocks for algorithms that are believed to provide quantum advantage. Some of these are of a more practical nature. For example, Qiskit users can build the circuit for Grover's algorithm by using `GroverOperator`.

Others are of more theoretical interest. For example, Fourier checking uses the `FourierChecking` method. (Click here to learn more about Fourier checking.)

Let's take a look at an example of the IQP circuit:

```
from qiskit.circuit.library import IQP

interactions = [[6, 5, 3], [5, 4, 5], [3, 5, 1]]
circuit = IQP(interactions)
circuit.decompose().draw("mpl")
```



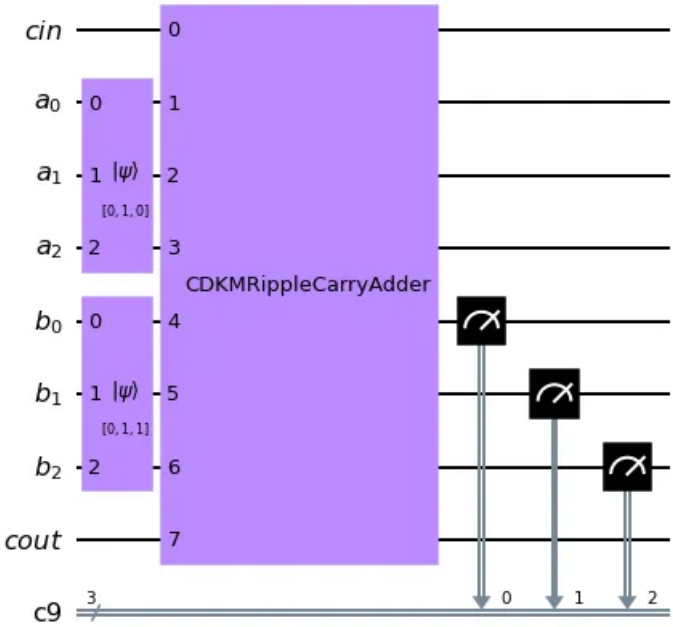
Arithmetic circuits

Some users may be surprised to learn that the circuit library includes numerous quantum circuits that are designed to perform classical arithmetic operations on qubit states or state amplitudes, such as addition or multiplication. These are useful for applications like amplitude estimation, which have the potential to be valuable in finance applications, and in algorithms like HHL, which is a proposed method for solving linear systems of equations.

As an example, let's try adding two 3-bit-long numbers, using a ripple-carry circuit to perform in-place addition (`CDKMRippleCarryAdder`). This adder take two numbers—A and B—then it adds them, and "sets" B with the result. In this example, A=2 and B=3:

```
from qiskit.circuit.library import CDKMRippleCarryAdder
adder = CDKMRippleCarryAdder(3)  # Create an adder of 3-bit-long numbers

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

# Create the number A=2
reg_a = QuantumRegister(3, 'a')
number_a = QuantumCircuit(reg_a)
number_a.initialize('010') # number 2 - equivalent to format(2, '03b')

# Create the number B=3
reg_b = QuantumRegister(3, 'b')
number_b = QuantumCircuit(reg_b)
number_b.initialize('011')  # number 3 - equivalent to format(3, '03b')

# Create a circuit to hold everything, including a classical register to put the result
reg_result = ClassicalRegister(3)
circuit = QuantumCircuit(*adder.qregs, reg_result)

# Compose setting A and B with the adder. Measure the result in B
circuit = circuit.compose(number_a, qubits=reg_a).compose(number_b, qubits=reg_b).compose(adder)
```

```
circuit.measure(reg_b, reg_result)
circuit.draw('mpl')
```



By calling the sampler in `qiskit.primitives.Sampler`, it is possible to simulate the execution of this circuit and observe that the result is 5.

```
from qiskit.primitives import Sampler

result = Sampler().run(circuit).result()
print(result.quasi_dists[0].keys())
```

```
dict_keys([5])
```

Standard gates

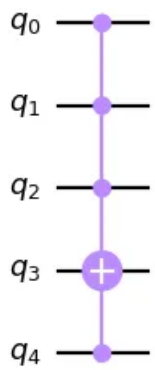
The **Standard Gates** section of the Qiskit Circuit Library is a foundational toolbox for building up a wide array of quantum circuits—from the humble X Gate all the way to more complex multi-controlled rotation gates.

Users can add standard gates to their circuits by calling the gates as methods—e.g., `circuit.x(3)` will append an X Gate to the qubit 3. That's the most common way to use them. With that being said, this is still an incredibly useful section of the circuit library, and of our documentation in general, as it provides some very useful information about the matrices underlying these gates.

Let's take a look at an example of how to append a multi-controlled CNOT to an existing quantum circuit, using the circuit library as a reference. In the following example we are constructing a multi-controlled X Gate (`MCXGate`) with 4 control qubits from the circuit library and applying it to a quantum circuit, specifying the control qubits are at indices 0,1,2 and 4:

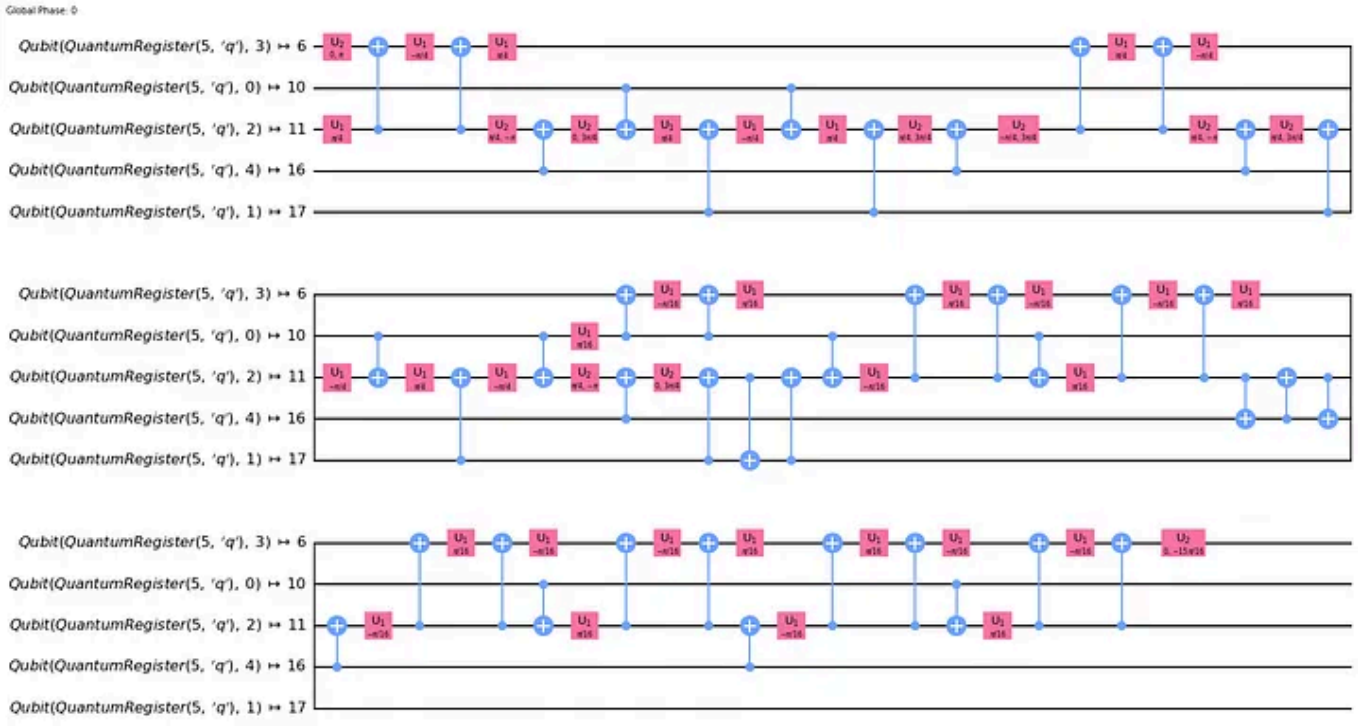
```
from qiskit import QuantumCircuit
from qiskit.circuit.library import MCXGate

# create circuit
circuit = QuantumCircuit(5)

# initialise MCX gate with 4 control qubits
gate = MCXGate(4)

# append gate to circuit, specifying qubit indices
circuit.append(gate, [0, 1, 4, 2, 3])
circuit.draw('mpl')
```

The resulting quantum circuit looks like this:



And here's an example of how we would transpile that to a quantum backend:

```
from qiskit import transpile
from qiskit.providers.fake_provider import FakeTokyo

backend = FakeTokyo()
transpiled = transpile(circuit, backend)
transpiled.draw('mpl', idle_wires=False)
```

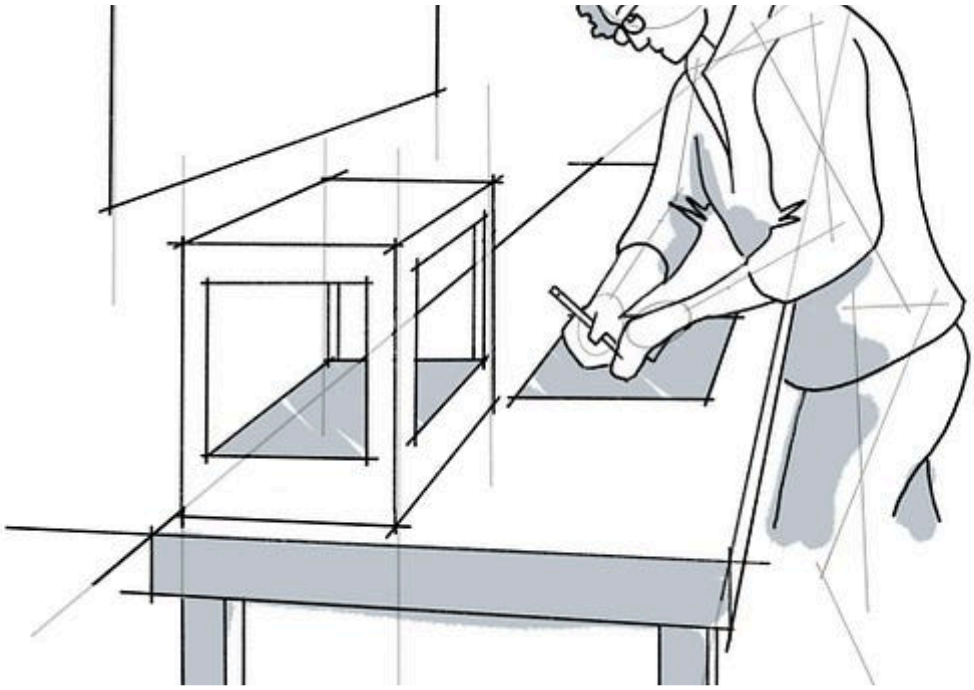


Other interesting circuits

This article only scratches the surface in terms of describing the many, many different types of circuits that are available in the Qiskit Circuit Library. Some other notable examples include the **quantum Fourier transform (QFT)**, which is an essential subroutine in many algorithms, **PhaseOracle**, which implements the oracles used in grover search, and **multi-control multi-target gate (MCMT)**, which makes it possible to (for example) implement an X gate acting on two qubits and controlled by three others.

Be sure to head over to the Qiskit Circuit Library to learn more about how to put these resources to use in your research.

Quantum Computing | Qiskit | Quantum Circuits | Quantum Gate | Variational Circuits



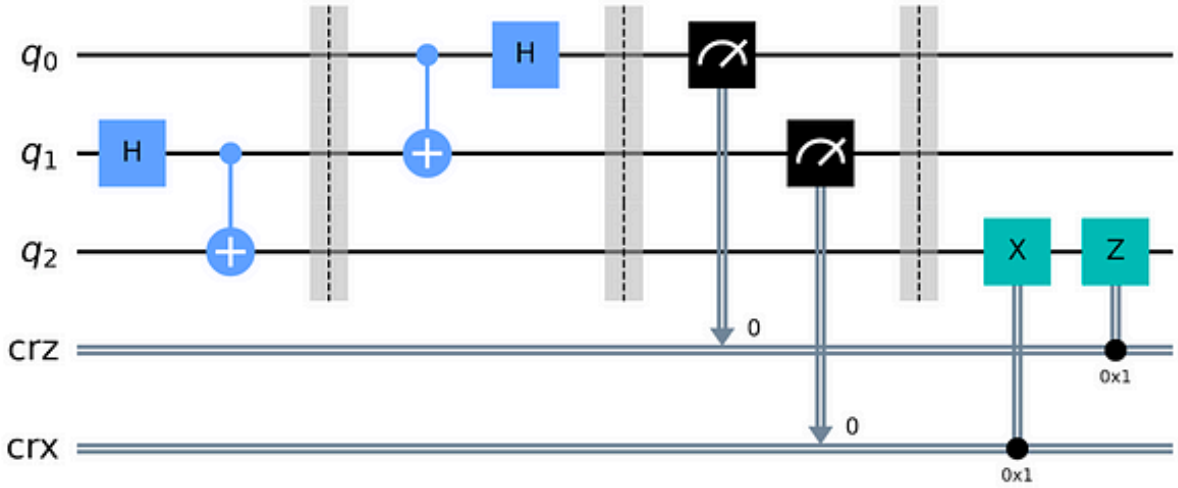
Qiskit in Qiskit

How Do I Build a Quantum Computer in My House?

By Ryan F. Mandelbaum, Senior Technical Writer, Qiskit

10 min read · Jun 22, 2020

1.2K 5



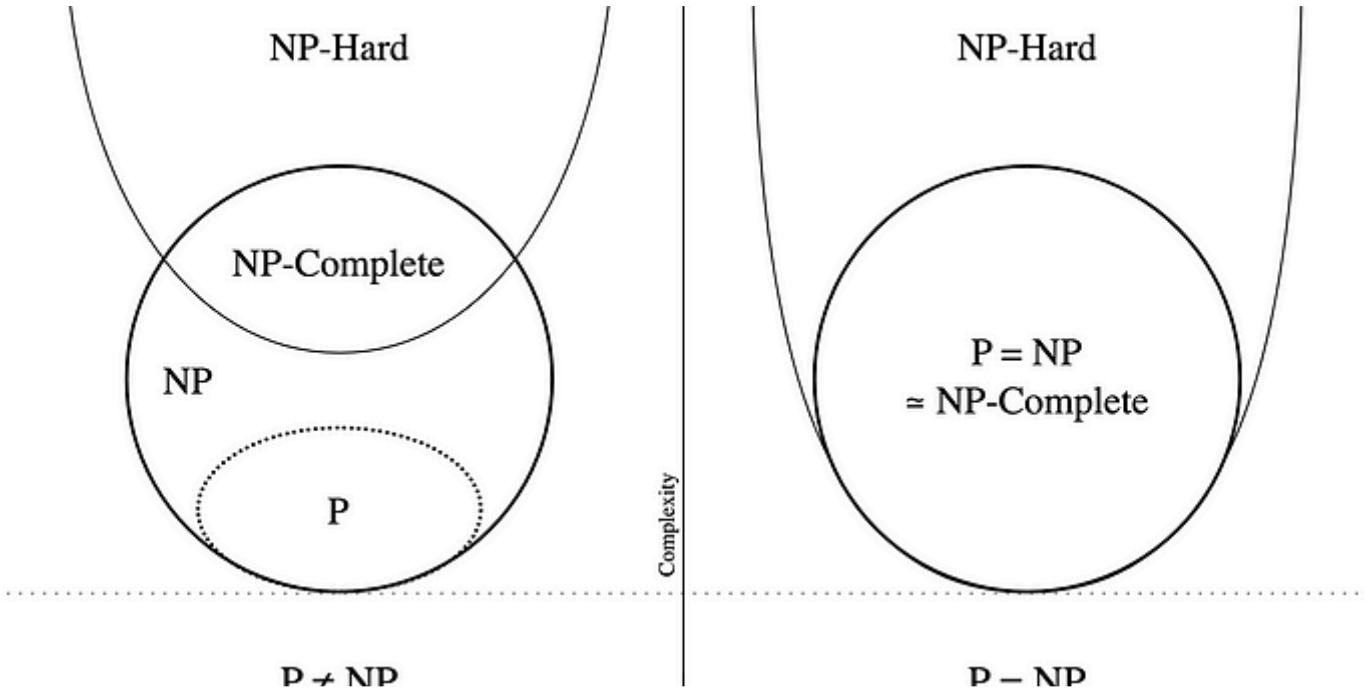
Qiskit in Qiskit

Learn Quantum Computing With These Seven Projects

By Ryan F. Mandelbaum, Senior Technical Writer, IBM Quantum and Qiskit

8 min read · Jan 6, 2021

469



Qiskit in Qiskit

What Can a Quantum Computer Actually Do?

By Bryce Fuller and Ryan Mandelbaum

9 min read · Feb 8, 2022

215 3

Qiskit in Qiskit

How The First Superconducting Qubit Changed Quantum Computing Forever

By Robert Davis, Technical Writer, IBM Quantum and Qiskit

11 min read · Sep 29, 2022

116

See all from Qiskit

See all from Qiskit

Recommended from Medium

Felix Xu

Geometrical Understanding of Quantum Computing

In this article, I am going to share my personal understanding of Quantum Computing, which is from the geometrical aspect.

7 min read · Nov 8, 2023

55 1

mohammed alaa

Quantum Computer Programming | (Part 1) :

Cirq: Google's Quantum Programming Framework & Quantum Machine Learning (QML) with TensorFlow Quantum (TFQ): Exploring Hybrid...

12 min read · Feb 1, 2024

Arsalan Pardesi

Quantum Fourier Transform, Quantum Phase Estimation and Shor's Algorithm

Using the Power of Quantum Computing to Solve Hard Problems—Part 4

6 min read · Dec 29, 2023

56

Waliur_Sun

Quantum Teleportation With Qiskit Code Implementation.

Introduction

4 min read · Jan 17, 2024

51

Qiskit in Qiskit

Qiskit v0.45 is here!

By Abby Mitchell, Luciano Bello, Matthew Treinish, Jake Lishman

9 min read · Nov 6, 2023

143

Holly Emblem in Towards Data Science

Quantum Deep Learning: A Quick Guide to Quantum Convolutional Neural Networks

Everything you need to know about quantum convolutional neural networks (QCNNs), including the benefits and limitations of these...

9 min read · Oct 4, 2022

321 1